Note to the researchers (remove this box before submitting). Every number and claim in this section was checked against the AutoVet source code and an actual test run on 31 October 2026. Text highlighted in yellow marks results that do not exist yet. Replace it with your real data, and never fill it with estimates.

- **System Testing:** fill in the Actual Result and Remarks columns of Table 3 after running each test case on the live system.
- **User Acceptance Testing:** fill in the number of respondents and the mean scores in Table 6. Make sure the interpretation scale matches the one in Chapter 3.
- **Performance Testing:** the text states that load testing was not conducted, which is true. If you measure response times before submitting, replace that limitation with your measured results.

SOFTWARE TESTING

The AutoVet Veterinary Clinic Management System was evaluated through six forms of testing: unit testing, integration testing, system testing, security and performance testing, user acceptance testing, and automated testing. Code consistency was also enforced across the codebase. Together, these tested the system at every level, from individual computations to the complete system as experienced by its intended users at Pet Wellness Animal Clinic. The unit and integration tests were written with PHPUnit through the Laravel framework's testing tools. Tests that require a database were executed against a dedicated MySQL test database (*autovet_test*) that was returned to a clean state after each test, so no test touched operational data. In the final test run on October 31, 2026, all 72 automated tests, comprising 432 assertions, passed.

1. Unit Testing

Unit testing verifies the smallest testable parts of a system in isolation from the database, the network, and the user interface. In AutoVet, unit testing focused on the computations behind the AI-assisted inventory forecasting module, implemented in the *ForecastMetrics* component. This module estimates each item's daily consumption with a least-squares linear regression on cumulative usage. It evaluates the estimate with the coefficient of determination (R^2), the root mean square error (RMSE), and the mean absolute error (MAE) under an 80/20 holdout. Because the module's outputs drive the stock-out warnings shown to clinic staff, its calculations were tested directly against values computed by hand.

Table 1

Unit Test Cases for the Inventory Forecasting Module

No.	Test Case	Expected Result	Result
1	Reproduce the worked accuracy example	Slope, intercept, R^2, RMSE, and MAE match values computed by hand	Passed
2	Project test days from the last training total	Holdout predictions start from the last known cumulative usage, consistent with the stock-out computation	Passed
3	Compute the baseline forecasting methods	Average, naïve, and seven-day moving average rates and errors match manual computation	Passed
4	Treat days without usage as zero	Baseline rates account for calendar days with no recorded usage	Passed
5	Score short histories in-sample	Items with fewer than ten data points are evaluated without a holdout split	Passed
6	Assign confidence labels	$R^2 \geq 0.70$ is labelled High, $R^2 \geq 0.40$ Medium, and lower values Low	Passed
7	Generate Poisson-distributed demand	Mean and variance of 20,000 draws approximate $\lambda = 1.6$	Passed
8	Framework sanity check	The test environment runs correctly	Passed

All eight unit tests passed, confirming that the forecasting module computes its regression estimates, accuracy measures, and confidence labels correctly.

2. Integration Testing

Integration testing verifies that separately developed components operate correctly when combined. AutoVet's integration tests were implemented as 64 Laravel feature tests. Each test sends an HTTP request to the application programming interface (API), which passes through routing, authentication, role-based middleware, validation, and the controller logic. The test then checks both the response and the resulting state of the MySQL database. This approach tests the interaction of the backend components as they operate in production, rather than in isolation.

Table 2

Summary of Integration Tests by Functional Area

Functional Area	Tests	Behavior Verified	Result
Access control	9	Role hierarchy is enforced: a veterinarian cannot create a clinic administrator, promote itself, or reset an administrator's password; a clinic administrator cannot create a super administrator	Passed
Authentication security	11	Identical responses for unknown e-mail addresses and wrong passwords; blocking of inactive, suspended, and archived accounts; revocation of sessions after a password reset	Passed
Tenant isolation	4	Staff cannot access another clinic's patients; notification channels are separated per clinic	Passed
Client portal restrictions	5	Client bookings are always created as pending; clients cannot approve their own appointments, open draft invoices, or write medical records	Passed
Appointments	3	Creation with a status, prevention of double-booking for the same pet, and status updates	Passed
Invoicing	3	Sequential invoice numbering and protection of finalized invoices from reverting to draft	Passed
Backups	2	Backups are stored per clinic and cannot be downloaded by another clinic	Passed
Data exposure	5	Clients receive only permitted settings and published content; the status endpoint discloses no database details; impersonation tokens are short-lived and audited	Passed
List and query validation	5	Page-size ceilings, whitelisted sort columns, literal search wildcards, and validated report parameters	Passed
Dashboard	3	Role-appropriate dashboard sections; failure of one section does not affect the others	Passed
Error handling	2	User-facing messages are shown while internal error details are withheld	Passed
API rate limiting	5	All API routes are limited to 300 requests per minute per signed-in user; staff and client accounts sharing an identifier are limited separately; signed-out visitors do not share a limit; login attempts are limited per account, so one locked account does not block other users	Passed
Whole-system endpoint check	1	Every API route (202 routes) is called as each of the five roles against a fully seeded clinic database; read requests use real record identifiers, and write requests with an empty body must be rejected with a client error; no route may return a server error	Passed
Demo data utilities	5	Generated data is labelled, leaves existing records untouched, and can be removed completely	Passed

Functional Area	Tests	Behavior Verified	Result
Application availability	1	The application responds successfully	Passed
Total	**64**		**Passed**

All 64 integration tests passed. The whole-system endpoint check made 1,010 requests (202 routes for each of five roles) without a single server error. This indicates that the system's components interact correctly and that its business rules, access restrictions, and data integrity constraints hold when requests pass through the complete backend.

3. System Testing

System testing evaluates the complete, integrated system against its specified requirements. AutoVet was tested as a whole in its deployed environment. The administrative web application and the client portal were hosted on Vercel. The backend API was hosted on Render, with a TiDB (MySQL-compatible) database and Supabase object storage. Black-box functional testing was used: for each module within the scope of the study, the researchers performed defined steps through the user interface and compared the actual outcome with the expected outcome. The modules covered were user authentication and role management, appointment scheduling, medical records, billing and invoicing, inventory management with AI stock forecasting, reports, the client portal, notifications, backups, and audit logging.

Table 3

System Test Cases

ID	Module	Test Procedure	Expected Result	Actual Result	Remarks
ST-01	Authentication	Log in as a veterinarian with valid credentials	The veterinarian dashboard is displayed	[actual result]	[Pass/Fail]
ST-02	Authentication	Log in with an incorrect password	An "Invalid credentials" message is shown and access is denied	[actual result]	[Pass/Fail]
ST-03	Appointments	Book an appointment through the client portal	The booking appears to staff with a Pending status	[actual result]	[Pass/Fail]
ST-04	Medical Records	Create a medical record for a consultation	All entered fields are saved and displayed in the pet's history	[actual result]	[Pass/Fail]
ST-05	Billing	Create an invoice with a service and a product	Subtotal, discount, and total are computed correctly	[actual result]	[Pass/Fail]
ST-06	Inventory / AI	Open the stock forecast of an item with usage history	Days until stock-out and the confidence level are displayed	[actual result]	[Pass/Fail]
ST-07	Reports	Generate a sales report for a date range	The report reflects transactions within the range	[actual result]	[Pass/Fail]
ST-08	Audit Logs	Update a record, then open the audit log	The change is recorded with the user and time	[actual result]	[Pass/Fail]

[Summarize the outcome here, e.g., the number of test cases passed out of the total, and any defects found and how they were resolved.]

4. Security and Performance Testing

Security Testing

Security testing examined whether AutoVet protects its data and functions from unauthorized access and misuse. The following controls were implemented and verified through the integration tests described in Table 2.

Table 4

Security Controls and Their Verification

Security Control	Implementation in AutoVet	Verification
Authentication	Token-based authentication using Laravel Sanctum; identical error responses for unknown accounts and wrong passwords to prevent account enumeration; blocking of inactive, suspended, and archived accounts; session revocation after a password reset	Authentication security tests (11)
Rate limiting	Login, 10 attempts per minute per account; registration and password recovery, 5 per minute per account; Google sign-in and e-mail verification links, 60 per minute; AI clinical support, 10 per minute; client e-mail notifications, 30 per minute; and all other API requests, 300 per minute per signed-in user. Login and recovery are counted per account because, behind the hosting proxies, all users reach the server from the same address	Route configuration; API rate limiting tests (5)
Authorization	Role-based access control for super administrators, clinic administrators, veterinarians, staff, and clients	Access control and portal tests (14)
Tenant isolation	Each clinic's records, notifications, and backups are inaccessible to other clinics	Tenant isolation and backup tests (6)
HTTP security headers	Strict-Transport-Security, Content-Security-Policy, X-Frame-Options (DENY), X-Content-Type-Options (nosniff), Referrer-Policy, and Permissions-Policy; removal of the X-Powered-By header	Data exposure tests
Input validation	Whitelisted sort columns, escaped search wildcards, page-size limits, and validated report parameters	List and query tests (5)
Dependency security	Automatic audit of frontend packages, failing on any high or critical vulnerability	npm audit in the CI pipeline

All security-related tests passed, indicating that the system enforces authentication, authorization, and data separation as designed.

Performance Testing

Several measures were built into AutoVet to keep response times acceptable as data grows. Database indexes were added for frequently searched fields and foreign keys. Frequently requested results are cached. List endpoints enforce a maximum page size. The dashboard retrieves all of its sections in a single request instead of several. However, formal load and stress testing, which measures response times under many simultaneous users, was not conducted within the period of the study. This is acknowledged as a limitation and is recommended for future work.

5. User Acceptance Testing (UAT)

User acceptance testing determined whether AutoVet meets the needs of its intended users and is acceptable for operational use. UAT was conducted at Pet Wellness Animal Clinic with three groups of respondents: veterinarians and clinic administrators, front desk staff, and clients (pet owners). A separate questionnaire was prepared for each group. Each questionnaire was based on ISO 9241-11:2018, which defines usability as the "extent to which a system, product or service can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use" (International Organization for Standardization [ISO], 2018). Each questionnaire had three sections: (1) general and demographic information, (2) a system evaluation organized by the three usability measures, and (3) open-ended feedback.

Table 5

Structure of the UAT Questionnaires

Respondent Group	Effectiveness	Efficiency	Satisfaction	Administrative Controls	Total Items
Veterinarians and Clinic Administrators	1–12	13–23	24–34	35–39[a]	39
Front Desk Staff	1–10	11–20	21–30	—	30
Clients (Pet Owners)	1–10	11–18	19–27	—	27

Note. [a]Answered by clinic administrators only.

Respondents rated each statement on a four-point Likert scale: 1 (Strongly Disagree), 2 (Disagree), 3 (Agree), and 4 (Strongly Agree). The responses for each section were summarized using the weighted mean:

$$\bar{x} = \Sigma fx / n$$

where f is the number of respondents who chose a rating, x is the rating value, and n is the total number of responses. Each mean was interpreted as follows: 3.26–4.00, Strongly Agree; 2.51–3.25, Agree; 1.76–2.50, Disagree; and 1.00–1.75, Strongly Disagree.

Table 6

Summary of UAT Results

Respondent Group	*n*	Effectiveness	Efficiency	Satisfaction	Overall Mean	Interpretation
Veterinarians and Clinic Administrators	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Front Desk Staff	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Clients (Pet Owners)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

[Discuss the results here: which usability measure scored highest and lowest for each group, the administrators' rating of the administrative controls, and recurring themes from the open-ended feedback.]

6. Automated Testing

Automated testing was implemented through a continuous integration (CI) pipeline in GitHub Actions. The pipeline runs automatically on every push and pull request to the project's main branches (*Seb1* and *main*), without manual intervention. It consists of three parallel jobs, summarized in Table 7. A failure in any step marks the entire run as failed, which prevents defective changes from passing unnoticed.

Table 7

Continuous Integration Pipeline

Job	Automated Steps
Backend (PHP 8.2)	Validation of the Composer configuration; dependency installation; syntax checking of every PHP source file; code style verification with Laravel Pint; execution of all 72 unit and integration tests against a MySQL 8.4 database service
Client Portal	Dependency installation; security audit of production dependencies (fails on high or critical vulnerabilities); static analysis with ESLint; type-checked production build
Administrative Application	Dependency installation; security audit of production dependencies; production build

During the evaluation, the pipeline itself was found to be failing: both frontend applications depended on package versions with published high-severity vulnerabilities, so the security audit step failed. The affected packages (*zod* and *follow-redirects*) were updated to patched versions, after which all checks completed successfully. A PHP dependency audit also found a medium-severity e-mail header injection flaw in *symfony/mime* (CVE-2026-45070), which was resolved by updating to version 7.4.22. In

the final verification on October 31, 2026, all 72 automated tests (432 assertions) passed, no code style violations or lint errors were found, no high-severity vulnerabilities remained, and both frontend applications built successfully.

Automated tests covered the backend, where the system's business rules, access control, and forecasting computations are implemented. The two React frontend applications were verified automatically through dependency audits, static analysis (client portal), and successful production builds, but they did not have automated user-interface tests. Their behavior was evaluated through system testing and user acceptance testing. In addition, the administrative application was exercised in a browser: the researchers signed in as a clinic administrator and opened all thirteen of its pages, none of which failed to render.

All of the corrections described in this section were deployed to the live system. The hosting services build and publish the application automatically from the project repository, and the changes were confirmed on the deployed backend and administrative application by inspecting their responses after deployment.

7. Code Consistency

Code consistency was maintained through automated formatting and static analysis tools, so that code written by different members of the team follows a single convention. The standards and tools used are summarized in Table 8.

Table 8

Code Consistency Standards and Tools

Component	Standard / Tool	What It Enforces
Backend (PHP, Laravel)	Laravel Pint (Laravel preset, based on PSR-12)	Indentation, spacing, brace placement, quote style, import ordering, and removal of unused imports
Backend (all files)	EditorConfig	UTF-8 encoding, LF line endings, four-space indentation, a final newline, and removal of trailing whitespace
Client Portal (TypeScript, React)	ESLint with typescript-eslint and React Hooks rules	Unused variables and imports, empty blocks, correct use of React hooks, and use of *const* where values are not reassigned
Client Portal	TypeScript strict mode	Type checking during every build
Whole project	Git attributes	Binary handling of PDF documents to prevent line-ending corruption

When these tools were first applied, 255 of 381 backend PHP files did not conform to the project's formatting standard. The client portal's lint configuration was missing, so static analysis could not run. When it was restored, 239 lint errors were reported, mainly unused imports and variables, empty blocks, and code left unused after earlier changes. All backend files were reformatted with Laravel Pint, and the full test suite was re-run to confirm that the formatting changed no behavior. All 239 portal lint errors were resolved, including the removal of unused code. Both checks were then added to the CI pipeline, so any future change that violates the formatting standard or introduces a lint error causes the pipeline to fail.

The administrative application has no linter of its own, so it was checked once with a set of ESLint correctness rules (undefined variables, rules of React hooks, undefined JSX components, unreachable code, and similar). Across 112 files, this found eleven problems that would fail at run time, in four files. The calendar page called a styling helper (*clsx*) that was never imported, so the page failed to render. The species list called a function that no longer existed when a species was clicked. The main layout called two React hooks after an early return, which made the application fail instead of showing the maintenance notice when maintenance mode was switched on during a session. An inventory insights component was missing its React imports. All eleven were corrected, and the same check now reports no problems.

Two limitations remain. First, the client portal still reports 182 lint warnings, most of them explicit uses of the *any* type for API responses that are not yet fully typed. These are reported as warnings so that they remain visible and can be reduced over time. Second, the administrative application does not yet run a linter in the CI pipeline; the correctness check above was a one-time review. Adding static analysis and full type definitions to both frontend applications is recommended for future development.

Summary

AutoVet was tested at the unit, integration, system, and acceptance levels. Eight unit tests confirmed the correctness of the AI forecasting computations, and 64 integration tests confirmed that the system's features, access restrictions, and security controls operate correctly on a real database. All 72 automated tests, with 432 assertions, passed and are executed automatically on every code change, together with code style and lint checks that keep the codebase consistent. Security controls were implemented and verified, and performance measures were built into the system, with formal load testing identified as future work. [Add one sentence each summarizing the system testing and UAT results.]

Reference

International Organization for Standardization. (2018). *Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts* (ISO Standard No. 9241-11:2018). https://www.iso.org/standard/63500.html